

CP Drid's Guide to Kubernetes Ingress Security: Hardening the Edge with Traefik and Ambassador

By CP Drid | Sec Fundamentals

Abstract: A hardened internal cluster means little if the front door stands open. This paper examines how to secure the Kubernetes ingress layer, the single entry point for external traffic, using Traefik and Ambassador as reference controllers. It covers the ingress controller as a security boundary, strict TLS termination and mutual TLS, edge authentication and authorization, rate limiting and web application firewall integration, layer 7 denial-of-service protection, safe secrets and certificate handling, least-privilege routing, and edge observability feeding a SIEM. It closes with a practical hardening checklist.

Introduction

In my previous writeup on securing east-west traffic with eBPF and Cilium, the focus was the internal conversation between pods. This paper turns to the perimeter, the point where external traffic enters the cluster. A hardened internal network means little if the front door stands open, and in Kubernetes that front door is the ingress controller. Configure it poorly and you hand an attacker a single, high-value target that fronts every service behind it. Configure it well and you gain a strong, centralized place to enforce encryption, identity, and traffic controls before a request ever reaches application code. Traefik and Ambassador serve as the two reference controllers throughout, but the principles hold for any controller you choose.

The Ingress Controller Is a Security Boundary, Not Just a Router

An ingress controller is the single entry point that routes external requests to internal services. That role makes it both a convenience and a liability. Because every external request flows through it, it is at once your best place to enforce policy and your largest attack surface. Treating it as plumbing is the mistake. It should be treated as a security boundary: sized, patched, and monitored accordingly. This is also why many teams migrated off the older NGINX ingress controller as it reached end of life, moving toward Traefik, Ambassador, and the emerging Gateway API. Whatever you run, keep the controller itself patched and running with least privilege, because a vulnerability in the front door exposes everything behind it. Scope the controller's own service account tightly, and do not grant it cluster-wide permissions it does not need to route traffic.

Enforce Encryption in Transit with Strict TLS Termination

The first control is strict TLS termination. Unencrypted HTTP should never cross into the cluster. Both Traefik and Ambassador automate certificate management through ACME and Let's Encrypt, which removes the usual excuse for expired or self-signed certificates. Automating issuance is only half the job. Security teams must also enforce the floor: reject anything below TLS 1.2, prefer TLS 1.3, disable weak cipher suites, and enable HSTS so clients refuse to downgrade to plaintext. For sensitive internal services, extend this to mutual TLS, where both the client and the server present certificates, so the ingress does not just encrypt traffic but also verifies who is on the other end. This maps directly to the encryption and public key infrastructure fundamentals every security program rests on.

Authenticate and Authorize at the Edge

Encryption proves the channel is private, not that the caller is allowed in. A hardened ingress enforces identity before traffic ever reaches a pod. Traefik does this through its ForwardAuth middleware, and Ambassador, built on the Envoy proxy, does it through external authorization filters. Both let you place an authentication check in front of your services, delegating to an OIDC or OAuth2 provider and validating tokens such as JWTs at the edge. The payoff is least privilege applied to north-south traffic: an unauthenticated request is rejected at the boundary instead of being trusted once it is inside. Centralizing authentication here also means you are not re-implementing it inconsistently in every microservice behind the controller, which is a common source of gaps.

Rate Limit and Filter Malicious Traffic

An ingress controller should defend the pods behind it, not merely forward to them. Two controls matter most. First, rate limiting: strict limits on API endpoints blunt brute-force credential attacks and absorb the early stages of volumetric denial-of-service attempts. Traefik provides rate-limiting middleware, and Ambassador integrates a dedicated rate-limiting service through Envoy. Second, web application firewall integration: pairing the controller with a WAF such as one running the OWASP Core Rule Set gives you inline inspection for injection, cross-site scripting, and the rest of the OWASP Top 10. Together, rate limiting and a WAF turn the edge from a passive router into an active filter that drops known attack patterns before they reach application logic.

Protect Against Layer 7 Denial of Service

Volumetric floods are not the only denial-of-service risk. Application-layer attacks exhaust resources with traffic that looks legitimate, so raw rate limits are not enough on their own. Set sane connection limits and request timeouts so a slow-drip attack cannot hold connections open indefinitely, a

technique often called slowloris. Cap request body and header sizes to stop oversized payloads from consuming memory. Where the controller supports it, enforce per-client concurrency limits in addition to per-endpoint rate limits. Both Traefik and Envoy expose these controls, and pairing them with an upstream content delivery network or edge protection service gives you a second layer that absorbs large floods before they ever reach your ingress.

Manage Secrets and Certificates Safely

Ingress security depends on private keys, so how you store them matters as much as how you use them. Keep TLS private keys and any authentication client secrets in the cluster secret store or, better, an external secrets manager, never baked into a container image or committed to a values file in source control. Restrict read access to those secrets to the ingress controller's service account alone. Rotate certificates automatically through ACME so a compromised or expiring key has a short useful life, and rotate any shared authentication secrets on a defined schedule. Treat the ingress namespace as sensitive and apply network policy so only the controller can reach the services it fronts.

Route with Least Privilege: No Wildcards, Strict Paths

Segmentation does not stop at the network layer, it extends to how you map routes. When you deploy or update a service, often by adjusting the Helm values file and refreshing the release, make sure the ingress routes are scoped tightly. Bind each route to the specific host and path a service actually serves. If an application lives under a defined path such as /app/reports, the ingress must restrict access to exactly that path and nothing broader. Wildcard and catch-all routing rules are a frequent cause of accidental exposure, quietly publishing services you never intended to reach the internet. Pair strict routing with namespace segregation and network policy so that even a misrouted request cannot cross into a namespace it has no business touching. This is least privilege applied to the front door.

Harden and Observe

Finally, close the loop with monitoring. The ingress controller sees every external request, which makes its access and error logs some of the highest-value security telemetry you have. Ship those logs to your SIEM, the same pipeline described in the previous writeup, and build detections on them: spikes in 401 and 403 responses that suggest credential stuffing, bursts of 429 responses that signal an attempted flood, or requests probing paths that do not exist. Combined with the runtime and network visibility already in place inside the cluster, edge logging completes the picture from the outside world all the way down to the pod.

Traefik or Ambassador

Both are strong choices, and the security principles apply equally to each. Traefik is a self-contained edge router known for its middleware model, native ACME automation, and straightforward configuration, which makes layering TLS, authentication, and rate limiting feel natural. Ambassador is built on the Envoy proxy, the same battle-tested data plane used in many service meshes, and leans on Envoy's mature filter chain for authorization, rate limiting, and observability. If you want simplicity and fast onboarding, Traefik is hard to beat. If you are standardizing on Envoy or need its advanced traffic-management features, Ambassador fits cleanly. Either way, the controller is only as secure as the policy you enforce on it.

Common Misconfigurations to Avoid

Most ingress compromises trace back to a handful of avoidable mistakes rather than exotic exploits. The controller's own dashboard or API is a frequent one: leaving the Traefik dashboard or an Envoy admin endpoint exposed to the internet hands an attacker a map of your routing and, sometimes, control of it. Overly permissive cross-origin resource sharing policies are another, quietly allowing untrusted sites to make authenticated calls on a user's behalf. Blindly trusting forwarded headers such as X-Forwarded-For lets a caller spoof source addresses and slip past IP-based rules, so only accept those headers from trusted upstreams. Watch for internal or administrative endpoints that were never meant to be public but end up reachable because a route was too broad. And never ship a controller or a fronted application with default credentials in place. None of these require a sophisticated attacker, which is exactly why they are worth a deliberate review.

Ingress Hardening Checklist

- Terminate TLS at the edge, reject below TLS 1.2, prefer TLS 1.3, and enable HSTS.
- Use mutual TLS for sensitive internal services.
- Automate certificate issuance and rotation through ACME.
- Enforce authentication at the edge via ForwardAuth or Envoy external authorization, backed by OIDC or OAuth2.
- Apply per-endpoint rate limits to blunt brute-force and flooding attempts.
- Integrate a WAF running the OWASP Core Rule Set.
- Set connection limits, request timeouts, and body and header size caps against layer 7 denial of service.
- Store TLS keys and secrets in a secret manager, restricted to the controller's service account.

- Bind every route to an explicit host and path, and forbid wildcard routing.
- Enforce namespace segregation and network policy around the ingress.
- Keep the controller patched and running with least privilege.
- Ship access and error logs to the SIEM and alert on 401, 403, 429, and unknown-path probing.

Secure your edge, validate your routes, and keep your ingress controllers locked down tight.

Stay secure. CPerk